

黄佳 - ClaudeCode实践 - 第2章 温故知新：记忆系统工程实践

前事不忘，后事之师。

上次技术讨论结束后，小冰迫不及待地在她负责的订单服务项目中尝试了Claude Code。她满怀期待地输入了第一个请求：“帮我写一个用户登录接口。”几秒后，Claude生成了一段看似完美的代码：基于Express框架，采用JavaScript语法，并附带了 `npm install` 命令，一气呵成。小冰盯着屏幕愣了3秒，才缓缓输入下一行：“我们的项目采用的是Fastify和TypeScript.....”随后，她耐心地纠正了模型的限制条件并重新描述了需求，终于获得了符合预期的代码。

第二天，小冰关闭终端，开启了一个全新的会话，让Claude编写订单创建接口。这一次，Claude表现得更加热情，洋洋洒洒地生成了完整的路由、控制器和数据模型，然而，框架依然是Express，包括管理器仍是npm，甚至连目录结构都与团队规范大相径庭。盯着屏幕上那行熟悉的 `const express = require('express')`，小冰心中涌起一股深深的无力感。

这种“AI失忆症”并非源于Claude的能力缺陷。恰恰相反，它在每个独立会话中都展现出了卓越的编程能力；然而，它所做出的每一个看似“合理”的选择——选用Express而非Fastify、使用npm而非pnpm、采用JavaScript而非TypeScript，却无一例外地撞上了团队规范的“红线”。

2.1 为什么AI需要“记忆”

小冰满腹牢骚地找到了咖哥。听完她的遭遇，咖哥并未急于给出解决方案。

“小冰，你还记得入职第一天的感觉吗？”咖哥问道。

小冰思索片刻：“当然记得，那时什么都不懂——代码仓库在哪里、分支策略是什么、CI流程如何运行.....事事都得请教同事。”

“没错。现在想象一下，如果你每天早上醒来，都会忘记前一天在公司学到的所有东西，每天都像是‘入职第一天’——你会怎么办？”

小冰不禁打了个冷战：“那不就是我现在用Claude的状态吗？”

“完全正确。”咖哥解释道，“对Claude而言，每一次对话都是它的‘入职第一天’（见图2-1）。这就好比一名技术过硬的新员工，首日上班却对公司技术栈一无所知，只能凭个人经验和行业惯例去猜测。或许猜得合乎逻辑，却未必符合实情。”



(图2-1 每一次对话都是Claude的“入职第一天”)

这是大模型的一个根本特征：会话间缺乏持久状态。每当你关闭一个Claude Code会话并开启新的会话时，Claude模型面对的都是一张完全空白的画布。它不记得你的项目采用何种框架，不记得你的团队偏好何种代码风格，甚至不记得5 min前你纠正过的错误。这并非某个特定模型的缺陷，而是当前所有大模型的共性——每一轮对话都必须从零开始构建上下文。

从软件工程的角度审视，这恰恰是“约定优于配置”(Convention over Configuration)原则的反面案例。Ruby on Rails之所以能赋予开发者极高的效率，归功于其预设的一套合理约定，例如，数据库表名采用复数、模型名使用单数、控制器置于app/controllers目录下，开发者只需要关注那些偏离约定的特殊部分即可。

然而，Claude的默认“约定”并非源自项目规范，而是源于其训练数据，即整个互联网公开代码的统计规律。在这一统计分布中，Express的出现频率高于Fastify，npm比pnpm更为普遍，JavaScript的样本量也远超TypeScript。因此，当缺乏具体的项目上下文时，Claude自然会倾向于选择最“主流”的方案。这些方案在通用场景下或许合乎逻辑，却未必契合你团队的特定规范。

换句话说，Claude缺失的并非能力，而是上下文。

既然如此，解决方案便水到渠成：既然每次对话都是Claude的“入职第一天”，我们不妨为其准备一份专属的“入职手册”，确保它在每次启动时自动研读。这份手册便是CLAUDE.md。作为一个纯 Markdown格式的文本文件，只需要将其置于项目约定的目录下，Claude Code在启动时便会自动扫描并加载其内容，将其作为系统上下文的一部分，预先注入对话的最前端。从此，Claude不再需要从零开始猜测项目的模样——一切规范与细节在手册中皆已清晰载明。

这一设计思路在软件工程中早已屡见不鲜。我们利用.editorconfig统一编辑器行为，借助.eslintrc规范代码风格，通过tsconfig.json约束TypeScript编译选项，依靠.prettierrc固化格式化规则。这些配置文件的本质，皆是将隐性的“团队约定”从人脑中外化为机器可读的声明式规则。

CLAUDE.md沿袭了同样的逻辑，唯一的区别在于其“读者”：不再是编辑器或格式化工具，而是一个具备自然语言理解能力的大模型。这正是“配置驱动”(Configuration-Driven)思维在AI时代的自然延伸与演进。

咖哥发言

或许有人会问：“Claude Code不是具备读取项目文件的能力吗？它完全可以自行查阅package.json来推断技术栈呀。”确实如此，但“主动探查”不仅会消耗宝贵的工具调用次数和上下文窗口，更关键的是，package.json中仅记录了依赖列表，却无法承载架构决策与团队约定等深层信息。CLAUDE.md的核心价值，正在于将散落在各处的隐性知识，凝聚为一份显性的指导文档，确保Claude在动手编码之前，便能构建起准确的项目心智模型。

2.2 五层记忆体系

“原理我明白了，那么CLAUDE.md究竟该放在哪里？”小冰追问道。

咖哥微微一笑：“问得好，答案是——它不只存在于一个地方。”

Claude Code的记忆系统采用了一种精妙的分层架构，从全局到局部共划分为5个层级。这种设计与操作系统的配置加载机制有着异曲同工之妙：正如Linux操作系统在读取配置时，会依次扫描/etc/目录下的全局配置、用户主目录下的个人配置，以及当前目录下的项目配置，通过层层叠加实现逐步细化；Claude Code的五层记忆体系也遵循着同样的设计哲学。

我们可以用一个贯穿始终的比喻来阐释这一体系：公司的制度文件层级。

第一层是集团总部发布的《集团行为准则》，这是全员必须遵守的铁律，任何分公司不得擅自修改，它确立了企业的核心价值观与底线。

第二层是各部门的《部门操作手册》，其效力覆盖部门内的所有项目，确保了同一业务线下的标准统一。

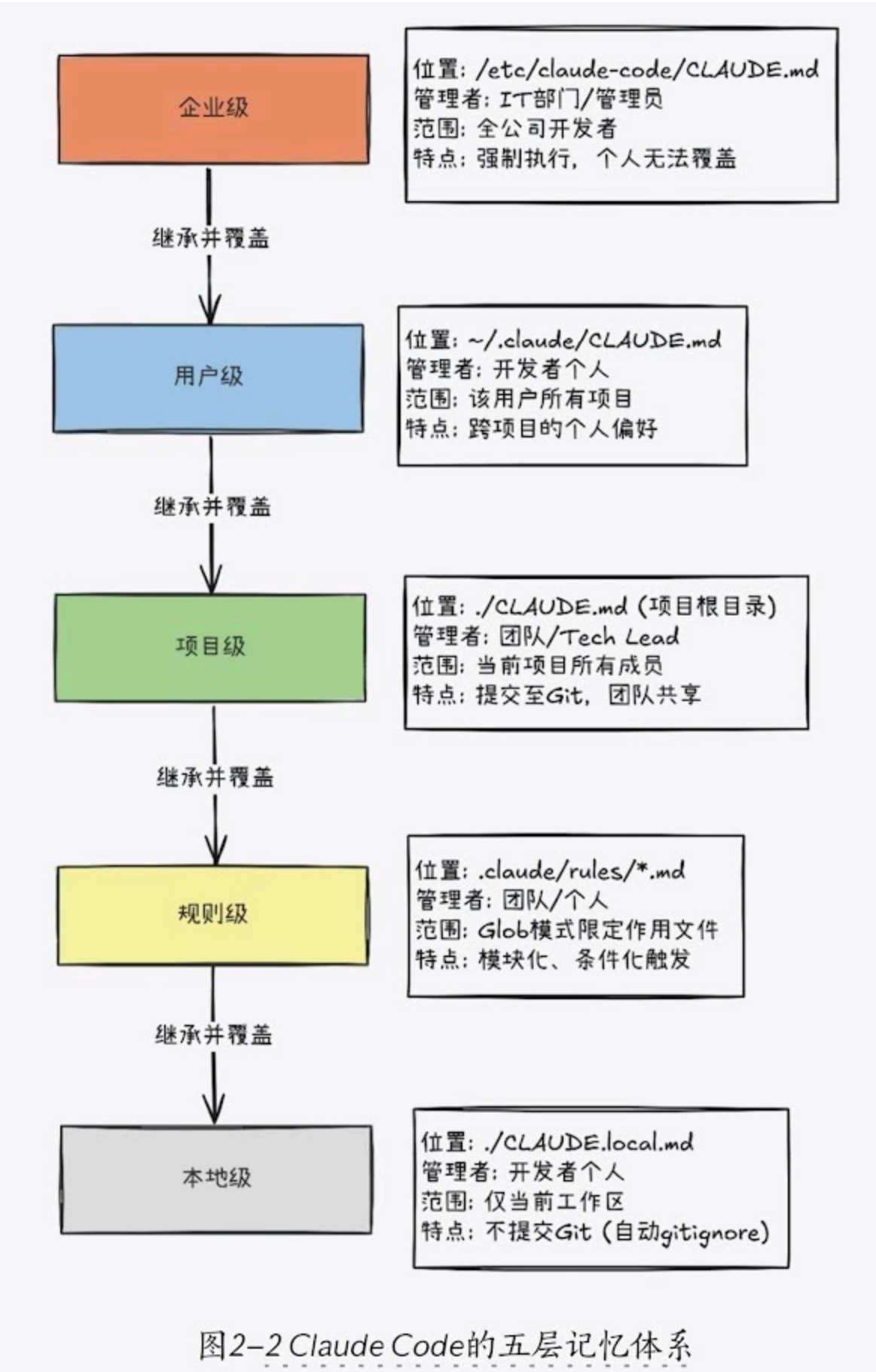
第三层是项目组的《项目开发规范》，它将规则细化到单个产品，明确了技术选型、编码风格等核心约定。

第四层是针对特定场景的《专项操作指引》。例如，《数据库迁移操作规范》仅在涉及特定任务时才被激活查阅，具有极强的场景针对性。

第五层则是每个员工工位上的私人备忘便签，这是完全个性化的知识沉淀，仅对自己可见，用于记录临时的调试技巧或个人偏好。

从上至下，适用范围逐渐收窄，内容颗粒度日益精细，管理灵活性也随之递增。

图2-2展示了Claude Code的五层记忆体系。



(图2-2 Claude Code的五层记忆体系)

接下来，我们逐层剖析各层级的职责与典型用法。

企业级构成了整个体系的“天花板”，也是开发者通常无法触及的层级。该层级由公司的IT管理员通过系统级配置文件（在Linux操作系统中通常为/etc/claude-code/CLAUDE.md）或Anthropic企业管理后台进行统一设定。其典型策略包括：禁止将代码发送至外部API，所有数据库操作必须经由ORM执行，严禁在代码中硬编码密钥等。这一级的存在，确立了企业级的合规底线：无论开发者在下4层如何配置，均无法绕过此处的强制约束。对个人开发者和小型团队而言，该层级通常为空白，不需要特别关注。

用户级相当于用户的“个人偏好档案”。该配置文件位于用户主目录下的~/.claude/CLAUDE.md，对在这台机器上打开的所有项目均生效。此处最适合存放那些与具体业务逻辑无关、但能体现个人工作习惯的通用偏好，如交流语言的选择（中文或英文）、Git提交信息的格式规范，或是将个人常用的快捷指令映射为具体的执行命令。这些偏好具有高度的稳定性，不会因项目的切换而改变。

```
<!-- ~/.claude/CLAUDE.md -->

# 个人编码偏好

- 交流语言：回复使用中文，代码注释使用英文
- 提交规范：Git提交信息格式遵循type[optional scope]: <description>
- 编程范式：优先采用函数式编程风格
- 快捷指令映射：
  - 当我说“执行测试”时，自动执行pnpm test
  - 当我说“启动服务”时，自动执行pnpm dev
```

项目级是日常开发中使用率最高的，也是本章的主角。它体现为项目根目录下的CLAUDE.md，必须提交至Git仓库，成为项目代码资产的一部分。

这意味着，当新同事克隆(clone)项目后，Claude Code会自动读取该文件，不需要任何额外配置，Claude即可瞬间获得完整的项目上下文。这好比新员工入职首日，在工位欢迎礼包中收到了一本专属的《项目开发手册》——翻开它，技术栈选型、目录结构、编码规范及常用命令便一目了然。关于如何精心撰写这份关键文件，我们将在2.3节中详细展开。

规则级是一项精巧的进阶设计，旨在解决“单一CLAUDE.md难以承载所有复杂规则”的痛点。你可以在.claude/rules/目录下部署多个独立的Markdown文件，使每个文件专注于一个特定主题（如数据库规范、API设计风格或测试策略）。更为关键的是，该层级支持条件化触发机制——规则文件仅在操作特定文件或进入特定上下文时才会被动态加载，从而避免了无关信息对上下文的冗余占用。关于这一机制的深度解析与实战用法，我们将在2.4节中专门探讨。

本地级是开发者的“私人便签”，对应CLAUDE.local.md。该文件不会被提交至Git仓库（Claude Code会自动将其纳入.gitignore规则），专为那些对个人极具价值但不宜共享的信息而设，如测试

服务器的地址与端口、个人调试过程中积累的独家心得，或是临时性的工作备忘录等。

例如，若知晓开发环境的数据库在特定时段会进行维护，此类信息只需要记录在本地层即可，不需要让全团队知晓，从而避免了信息噪声。

```
<!-- ./CLAUDE.local.md (自动加入 .gitignore) -->

# 本地开发备忘

- 数据库连接: postgresql://dev:dev123@10.0.1.50:5432/orders_dev
- 维护窗口: 每周三22:00-23:00进行数据库维护，严禁在此时段执行迁移操作
- 故障排除: 遇到CORS报错时，优先检查Nginx配置(/usr/local/etc/nginx/nginx.conf)
- 联调配置: 前端服务运行于5173端口，需要手动配置代理转发
```

小冰听到这里，敏锐地提出了一个关键问题：“若这5个层级的内容发生冲突该如何处理？例如，我在用户级设定了‘4空格缩进’，而项目级却规定了‘2空格缩进’。”

“问得好。”咖哥赞许道，“这些文件之间遵循的是‘叠加而非覆盖’的原则。Claude Code会读取所有层级的内容，将其合并为一份完整的上下文。当出现矛盾时，Claude模型会基于其判断力进行决策，而其核心逻辑是具体性优先。在缩进的例子中，项目级的‘2空格’规则针对当前特定项目，其具体性高于用户级的通用‘4空格’偏好，因此Claude会优先采纳前者。然而，企业级是个例外。它拥有绝对强制力，无论下层如何配置，都无法绕过其约束。这就好比公司的法务红线或安全底线，任何项目经理或个人偏好都无权逾越。”

值得一提的是，CLAUDE.md还内置了强大的**文件引用机制**。在任何层级的记忆文件中，均可使用 `@path/to/file` 语法引入其他文件的内容，从而实现配置的模块化组织。

例如，在项目层的CLAUDE.md中写入 `@docs/api-conventions.md`，Claude便会自动加载并解析API约定文档的内容。该机制支持最多五层的嵌套递归，足以应对绝大多数复杂项目的组织需求。

这为大型项目提供了一种优雅文档治理方案：CLAUDE.md不再需要承载所有细节，而是作为一份“索引总纲”，灵活指向各个专题文档。这种设计成功避免了将海量信息堆砌在单一文件中的弊端，让配置结构更加清晰、可维护。

2.3 CLAUDE.md写作范式

理解了记忆文件的存储位置与层级职责后，更核心的命题浮出水面：如何撰写？

咖哥在白板上郑重写下4个大字：**少即是多**。

“CLAUDE.md的内容具有全局持久性，”咖哥解释道，“它会在每一次对话初始化时，被完整注入上下文窗口中。这意味着，你写的每一行文字，都会在每一次交互中消耗宝贵的Token配额。”

他打了一个生动的比方：“你可以将上下文窗口想象成一张空间有限的办公桌。桌面的面积是固定的。如果你在Claude Code开始实际工作之前，就在桌上堆满了厚厚的参考资料（即冗长的规则文件），那么留给真正需要处理的代码、文件和临时草稿的空间会所剩无几。一旦桌面被占满，Claude Code就不得不‘扔掉’一些早期的关键信息以腾出空间，这往往导致它‘遗忘’重要的指令或上下文细节。”

这个比喻不仅形象，更有坚实的数据支撑。根据Anthropic的工程实践数据，Claude Code 的 System Prompt（系统提示词）已内置了约50条核心指令——这些构成了模型正常运行的“操作系统级”基石。

当用户自定义的CLAUDE.md内容加入后，若总指令数突破150条的临界值，模型对指令的遵循质量便会呈现显著衰减。其背后的逻辑是：指令密度与遵循概率成反比。指令越冗长繁杂，每一条规则被模型“深度关注”和“严格执行”的概率就越低。

这与管理学中的“信息过载”(Information Overload)原则不谋而合：当员工手册厚到无人能通读时，其实际执行效果往往不如一张提炼了核心精髓的A4纸摘要。在AI协作中，精简不仅是节省Token的手段，更是确保指令有效落地的关键策略。

那么，究竟什么内容值得写入CLAUDE.md？咖哥给出了一个高效的“三问框架”作为筛选标准。

第一问：WHY（为什么这样做）

核心价值：赋予Claude模型“举一反三”的推理能力。

这类信息旨在揭示决策背后的底层逻辑，帮助Claude理解项目的核心关切。一旦掌握了“为什么”，即便遇到未被明确规定的边缘场景，模型也能基于原则进行正确推导。

示例：“我们选择Fastify而非Express，因为Fastify的schema-based validation与本项目‘类型安全优先’的战略一致。”

效果：当Claude理解了“类型安全”是最高优先级后，即便你在某处未指定库，它也会主动倾向于选择类型定义更完善的方案，而非仅仅机械地匹配关键词。

第二问：WHAT（做什么、不做什么）

核心价值：划定不可逾越的“行为边界”。

这是最直接、最刚性的行为指令，明确界定允许与禁止的范围。此类规则通常不需要解释原因，要求的是绝对的服从与执行。绝大多数项目规范属于此列。

示例：“必须使用pnpm作为包管理器；禁止使用npm或yarn。”

效果：建立清晰的“红线”，消除模型在工具选型上的不确定性，确保技术栈的统一性。

第三问：HOW（如何一步步做）

核心价值：固化“标准作业程序”。

这类内容适用于常用命令、固定流程或特定操作范式。模型不需要理解命令背后的复杂原理，只需要准确执行既定的步骤即可。

示例：“执行数据库迁移请运行`pnpm db:migrate`；填充测试数据请运行`pnpm db:seed`。”

效果：将团队的最佳实践转化为模型的“肌肉记忆”，减少命令拼写错误或参数遗漏导致的低级失误，提升执行效率。

并非所有信息都需要同时回答这3个问题。大多数规则只需要在WHAT层面用一句话明确界定即可。只有那些容易被误解或者模型容易“好心办坏事”的关键决策，才值得补充WHY层面的深度解释。

如果你发现在自己的CLAUDE.md中，每一条规则后面都附带了一大段背景阐述或情感抒发，那通常意味着内容过载了。精简，才是高效协作的王道。

下面通过一个反面案例与后续的正面范例对比，帮助你快速建立直觉。

先来看一个反面案例：模糊的“正确的废话”。

项目说明

这是一个电商平台的前端服务，我们团队有5个人，项目从2024年3月开始开发。

使用Node.js编写，是一个RESTful API服务。

代码要写得好一些，注意性能。

测试要全面。

请遵循最佳实践。

这份CLAUDE.md充满了人类社交语境下的客套话和模糊指令，对AI而言，几乎等同于无效信息。

- “写得好一些”：好到什么程度？是符合Prettier格式，还是运用了设计模式？
- “全面”：覆盖到哪些场景？是单元测试、集成测试，还是E2E测试？覆盖率要求是80%还是100%？
- “最佳实践”：是谁的最佳实践？

Claude读完这段话，获得的可操作信息几乎为零。

尤其致命的是“遵循最佳实践”这条指令。它看似正确，实则空洞无物。因为Claude默认会尝试遵循它所认知的“最佳实践”（基于互联网海量数据训练的统计规律）。问题的核心恰恰在于：模型通用的“最佳实践”与你团队特定的“技术约定”之间往往存在偏差。

此外，“团队有5个人”“2024年3月开始开发”这类元数据，对代码生成逻辑没有任何指导意义，却白白占用了宝贵的上下文空间。

再看一个正面范例：精准的“操作手册”。

```
# 订单服务 API

## 技术栈
- Node.js 20 + TypeScript 5.3 (严格模式)
- Fastify 4 框架 (不使用 Express)
- Prisma ORM + PostgreSQL 15
- pnpm 8 包管理 (不使用 npm/yarn)

## 项目结构
- src/routes/ - 路由定义, 只做参数解析和响应构造
- src/services/ - 业务逻辑层, 所有核心逻辑在此
- src/repositories/ - 数据访问层, 封装Prisma调用
- src/schemas/ - Zod 验证 schema, 与路由一一对应

## 关键约定
- API 统一返回格式: { success: boolean, data?: T, error?: { code: string, message: string } }
- 错误码使用UPPER_SNAKE_CASE, 如 ORDER_NOT_FOUND
- 数据库表名snake_case复数形式, 主键UUID, 必带created_at和updated_at

## 常用命令
pnpm dev          # 启动开发服务器, 端口 3000
pnpm test         # 运行全部测试(vitest)
pnpm build        # TypeScript编译+类型检查
pnpm db:migrate   # 执行Prisma数据库迁移
```

两份文件的差距一目了然。优秀的版本简洁、具体且具备高度可操作性，每一条规则都能直接锚定Claude模型的行为逻辑。

- 技术栈锁定：看到“Fastify 4框架（不使用Express）”，Claude会立即排除Express相关方案，直接生成基于Fastify的代码。
- 架构规范落地：看到目录结构说明后，Claude能精准判断新路由文件应置于src/routes/目录下，而非随意堆砌。
- 执行零误差：看到常用命令及其注释后，Claude能准确执行构建与测试任务，杜绝了在npm run build还是yarn build之间盲目猜测的可能。

除手动编写以外，Claude Code还提供了两个便捷工具来辅助管理记忆文件。

第一个是 /init 命令。在项目根目录执行该命令后，Claude Code会自动扫描项目的目录结构、依赖配置文件（如package.json、pyproject.toml、Cargo.toml等）以及其他关键配置，随即生成一份CLAUDE.md初稿。这份初稿通常涵盖技术栈识别、目录结构概览和常用脚本命令，是一个极佳的起步基础。

然而，仍需要手动审查并补充/init无法自动推断的内容，如分层架构的具体约束、API响应格式的统一约定、团队特有的命名规范等。

这些“隐性知识”仅存在于团队成员的脑海中，无法单纯从配置文件中提取，必须通过人工干预将其显性化，才能确保AI的行为完全符合团队预期。

第二个是 `/memory` 命令，用于在对话过程中动态更新记忆。执行该命令后，Claude Code会弹出文件选择器，列出当前所有可用的记忆文件供你选择编辑。这些文件覆盖了不同的作用范围，具体如表2-1所示。

表2-1 项目记忆的作用范围

文件类型	路径	作用范围
项目记忆	<code>./CLAUDE.md</code> 或 <code>./.claude/CLAUDE.md</code>	团队共享：可纳入版本控制，适用于统一团队规范
用户记忆	<code>~/.claude/CLAUDE.md</code>	个人全局：个人偏好配置，对当前用户的所有项目均生效
项目本地记忆	<code>./CLAUDE.local.md</code>	个人局部：仅针对当前项目的个人偏好，通常会自动被.gitignore排除，避免提交至代码仓库

选中文件后，系统将在编辑器中直接打开它，供你即时修改。

假设你在交互中发现Claude Code再次误用了moment.js，而项目规范明确要求使用date-fns。此时，你不需要退出对话或手动查找文件，只需要执行以下步骤。

第1步，输入 `/memory` 命令。

第2步，选择项目级的CLAUDE.md。

第3步，追加规则：日期处理统一使用date-fns，禁止使用moment.js。

第4步，保存并关闭。

下次对话启动时，这条新规则即刻生效。若该偏好仅属于个人习惯而不愿影响团队，可选择写入CLAUDE.local.md——该文件默认被.gitignore排除，不会提交至代码仓库。

这一机制构建了一个“犯错→纠正→记忆写入→避免再犯”的良性闭环。随着时间的推移，你的记忆文件将愈发精准，Claude Code在项目中的表现也将随之持续进化。

2.4 条件化规则系统

随着项目的演进，单一CLAUDE.md终将面临容量瓶颈。

当项目涵盖前端组件规范、后端API设计、数据库迁移流程、测试策略（覆盖率/mock）以及基础设施（Docker和CI/CD）等全部规则时，如果将所有内容堆砌在一个文件中，将引发两大核心问题。

- 维护噩梦：文件变得极度冗长，查找和更新特定规则变得困难。
- 注意力分散：例如，当Claude编写一个简单的前端组件时，其上下文被加满了数据库迁移规范和CI配置要求等无关信息，这些噪声会稀释模型的注意力，导致其在处理当前任务时效率下降，甚至产生幻觉或忽略关键的前端约束。

这正是`.claude/rules/`目录的核心价值所在。它将记忆系统从“一本大而全的厚重手册”进化为“一个按需取用的模块化知识库”。

该机制具备两大核心优势。

领域拆分：允许将庞大的规则集拆解为多个主题文件，每个文件专注于特定领域，极大地提高了可维护性。

智能激活：通过在文件头部添加YAML Frontmatter，利用`paths`字段声明Glob模式；只有当Claude Code操作的文件路径匹配该模式时，对应的规则才会被激活并注入上下文；在不匹配的场景下，这些规则文件安静地存储在磁盘上，不消耗任何token或上下文空间。

来看一个示例。若需要规范测试文件的编写，可创建如下规则文件。

```
<!-- .claude/rules/testing.md -->
---
paths:
  - "**/*.test.ts"
  - "**/*.spec.ts"
  - "tests/**"
---

# 测试规范

- 采用vitest作为测试框架，禁用jest
- 每个测试文件必须包含describe代码块，且其名称需要与被测模块保持一致
- 模块模拟统一使用vi.mock()，禁止手动Mock
- 异步测试一律采用async/await语法，禁止使用done回调
- 测试数据需要通过factory函数生成，严禁在测试中硬编码
```

在此配置下，当Claude编辑`src/services/order-service.ts`时，该测试规范不会被加载；而一旦转向编写`tests/order-service.test.ts`，这些规则即刻自动生效。这种“按需加载”(Lazy Loading)理念一脉相承：不需要在启动时预载所有潜在资源，仅需要在真正需要时动态调用。

同样的思路可推广到其他开发领域。例如，数据库规范仅在操作Prisma文件或数据访问层时加载，而API设计规范则仅在编辑路由或Schema文件时生效。

```
<!-- .claude/rules/database.md -->
---
paths:
  - "prisma/**"
  - "src/repositories/**"
---

# 数据库规范

- 迁移文件一旦提交至main分支，严禁修改，仅运行创建新迁移
- 所有查询必须经由repository层，service层禁止直接调用Prisma客户端
- 批量操作必须包裹在事务中，单次写入超过100条记录时强制分批处理

<!-- .claude/rules/api-design.md -->
---
paths:
  - "src/routes/**"
  - "src/schemas/**"
---

# API 设计规范

- 每个路由必须配置对应的Zod schema以进行入参验证
- 列表接口统一支持分页参数：page（从1开始计数）和limit（默认20，最大值100）
- 错误响应结构必须包含机器可读的code字段与人类可读的message字段
```

小雪听罢，精辟地总结道：“这好比为企业不同岗位的员工量身定制了专属操作手册——前台遵循前台流程，仓库恪守仓库规范，不需要人人背诵全公司的制度。唯有当你步入仓库作业时，才需要翻开那本《仓库管理手册》。”

咖哥点头补充：“这种模块化的规则组织方式另有一大实效：在多人协作场景下，各领域专家可独立维护其专属规则文件，从而大幅降低Git合并冲突的概率。前端工程师专责frontend.md，后端工程师维护api-design.md，DBA把控database.md——各司其职，互不干扰。随着团队规模的扩张，这种‘分而治之’策略的价值将愈发凸显。”

咖哥发言

若规则文件未配置paths前置元数据，系统将其视为全局无条件规则，在每次会话中强制加载——其效果等同于直接写入CLAUDE.md主文件。因此，务必为每个规则文件设定精准的paths限定，方能真正发挥记忆系统“按需加载”的核心优势。

2.5 实战：3种典型项目配置

理论阐述至此，让我们通过3种典型项目的CLAUDE.md实战写法来融会贯通。请留意它们在侧重点上的显著差异。

- 前端项目：围绕组件规范与状态管理策略展开。
- 后端项目：聚焦分层架构设计与API约定。
- 数据项目：强调数据处理流程严谨性与实验复现规范。

这些差异绝非偶然，它们精准反映了不同技术领域中“最高频出错点”的分布。优秀的CLAUDE.md应当像精确制导武器，将火力集中覆盖在那些最容易引发问题的关键地带。

2.5.1 React前端项目配置

在React前端项目中，组件粒度划分、状态归属以及样式方案选择往往是分歧的“重灾区”，也是Claude最容易“自作主张”导致代码风格割裂的地方。若未明确指定状态管理策略，Claude极有可能在同一项目中混用Redux、Context API和本地State，造成架构混乱。

以下是一份针对电商平台前端项目的CLAUDE.md实战范例。

```
# 电商平台前端项目规范

## 技术栈
- 核心: React 18+TypeScript (严格模式)
- 构建/包管: Vite 5 +pnpm
- 状态管理:
  - 服务器状态: TanStack Query(React Query)
  - 客户端全局状态: Zustand
- 样式方案: Tailwind CSS (严禁使用CSS Modules或styled-components)

## 组件规范
- 范式: 仅使用函数组件+Hooks, 禁止class组件
- 命名:
  - 文件采用PascalCase (如ProductCard.tsx)
  - Props 类型命名为组件名 + Props (如 ProductCardProps)
- 目录结构:
  - 通用UI原子组件 -> src/components/ui/
  - 业务领域组件 -> src/components/features/

## 状态管理决策树
- 服务器数据 (API 响应、缓存、同步) ->TanStack Query
- 全局客户端状态 (主题切换、用户登录态、购物车临时态) ->Zustand
- 组件局部状态 (表单输入值、Dropdown展开/折叠、Modal显示) ->useState

## 常用命令
```bash
pnpm dev # 启动开发服务器, 端口 5173
```

```
pnpm build # 生产环境构建
pnpm test # 运行vitest 单元测试
pnpm lint # ESLint 代码检查
```

请注意其中“状态管理决策树”这一部分，它不仅仅是罗列技术栈名称，而是构建了“场景到工具的映射逻辑”。这种“决策指导”远比单纯的工具清单有效得多——它直接告诉Claude在遇到具体问题“该选哪把钥匙”，从而在源头上杜绝了架构风格的漂忽问题。

## 2.5.2 Node.js后端项目配置

Node.js后端项目的核心命脉在于分层架构的职责边界与数据格式的统一。后端代码的“正确性”往往取决于各层之间是否严守依赖规则——如果route层直接调用Prisma，绕过了service层和repository层，业务逻辑便会泄漏到基础设施层，导致后续维护陷入泥潭。

以下是一份针对订单微服务的CLAUDE.md实战范例。

```
订单微服务后端规范

技术栈
- 运行时: Node.js 20+TypeScript 5.3
- 框架: Fastify 4 (严禁使用 Express)
- 数据库: Prisma ORM + PostgreSQL 15
- 工具链: pnpm (包管理), Zod (数据验证)

分层架构 (严格单向依赖)
- routes/ -> 解析请求参数、校验输入、调用controller、构造HTTP响应
- controllers/ -> 编排service调用、处理HTTP语义、异常捕获与转换
- services/ -> 核心业务规则实现、事务控制、多repository协调
- repositories/ -> 封装Prisma查询、数据映射 (严禁其他层级直接调用Prisma)

API 响应标准
所有接口必须统一返回格式: { success: boolean, data?: T, error?: { code: string, message: string } }
分页规范: page (从1开始), limit (默认20, 最大值100)
错误码: 必须使用UPPER_SNAKE_CASE格式

常用命令
```bash
pnpm dev      # 启动开发服务器, 端口 3000
pnpm test     # 运行vitest测试套件
pnpm build    # tsc 编译
pnpm db:migrate # 执行Prisma 数据库迁移
pnpm db:studio # 启动Prisma Studio 可视化界面
```

在此配置中，最具核心价值的一条规则莫过于“严禁其他层级直接调用Prisma”。这绝非单纯的代码风格偏好，而是一条不可逾越的架构铁律，它直接决定了项目的可维护性与生命力。

2.5.3 Python数据项目配置

Python数据项目呈现出与Web开发截然不同的面貌。若Claude不了解数据科学领域的工程习惯，极易生成违背最佳实践的代码，例如，滥用`inplace=True`来修改`DataFrame`，破坏了数据流的可追溯性；或者，混用`None`与`pd.NA`，导致缺失值处理逻辑混乱。

以下是一份针对用户行为分析系统的CLAUDE.md实战范例。

```
# 用户行为分析系统规范

## 技术栈
- 环境: Python 3.11+uv (管理依赖与虚拟环境)
- 核心库:
  - 数据处理: pandas 2.x
  - 机器学习: scikit-learn
  - 可视化: matplotlib+seaborn
- 代码规范:
  - 类型标注: 严格使用typing模块
  - 文档字符串: 强制采用Google Style

## 项目结构
- notebooks/: 探索性数据分析(Jupyter), 命名格式为"序号-描述.ipynb", 如01-数据探索.ipynb
- src/data/: 数据加载、清洗与管道构建
- src/features/: 特征工程逻辑
- src/models/: 模型定义、训练流程与评估指标
- tests/: pytest 单元测试套件

## 数据处理约定
- 缺失值: 统一使用pd.NA, 禁止使用None或裸用的np.nan
- 日期列: 统一转换为datetime64类型, 通过输出/存储格式为YYYY-MM-DD
- 内存优化: 低基数的分类变量必须显式转换为category类型
- DataFrame: 禁止使用inplace=True, 所有转换操作必须返回新的DataFrame副本

## 常用命令
```bash
uv sync # 安装/同步依赖环境
uv run pytest # 运行测试套件
uv run jupyter lab # 启动Jupyter Lab
uv run python -m src.train # 执行模型训练脚本
```

咖哥指着3份配置文件，做出了最后的总结点评：

“你们注意到了吗？这3份文件的篇幅均控制在二三十行，但字字珠玑，每一行都是‘干货’。它们摒弃了Claude早已熟知的通用知识——你不需要向它解释什么是React，也不必赘述Python的缩进规则——而是精准聚焦于特定领域与本项目独有的约定和抉择。”

这正是CLAUDE.md写作的核心法则：**只记录那些Claude必须知晓、却无法自行推断的关键信息。请牢记一条黄金检验标准：如果删去某条规则后，Claude依然能做出正确的行为，那么这条规则就不该出现在CLAUDE.md中。**

## 本章小结

回到本章开篇的场景：小冰每次开启全新会话时，Claude总是习惯性地用Express替代Fastify，用npm替代pnpm。现在我们明白，这并非Claude模型的能力缺陷，而是因为它缺失了一份关键的“入职手册”。

CLAUDE.md正是解决这一痛点的关键机制。它将团队的技术选型、编码规范、项目结构以及常用命令，固化为一份Claude可读的配置文档。每当会话启动时，Claude Code便会自动加载这些信息，从而彻底跳过那些反复纠正的无效沟通，直接进入高效协作状态。

五层记忆体系的设计，深刻体现了软件工程中“关注点分离”的经典原则。

- 企业级：严守安全底线，由组织统一管理。
- 用户级：沉淀个人偏好，尊重个体差异。
- 项目级：承载团队共识，确保协作一致。
- 规则级：实现规则的模块化与条件化加载。
- 本地级：容纳私人备忘，提供灵活的临时上下文。

每一级均拥有明确的职责边界与管理主体，既有力支撑了团队协作，又充分尊重个人习惯。

其中，条件化规则系统更是一项精巧的工程设计。它利用Glob模式精准限定规则的作用域，实现了“在恰好的时刻，将恰当的信息投递给上下文中”的智能化机制。这种设计彻底摒弃了将所有规则一股脑倾倒在Claude面前的粗放模式，确保了上下文的纯净与高效。

关于写作范式，“少即是多”绝非一句空洞的口号，其背后有着严肃的技术原理支撑。

上下文窗口是模型最稀缺的战略资源，每一个Token都承载着高昂的机会成本。

将无关信息强行塞入CLAUDE.md，无异于在高峰期的地铁车厢里硬塞进不需要乘车的乘客——这不仅占用了宝贵的空间，更导致真正需要上车的“关键信息”被挤在门外，无法进入模型的视野。

一份优秀的CLAUDE.md应当像一份精炼的作战备忘录，严格遵循以下3条铁律。

- 必要性：每一条都是Claude必须知道且无法自行推断的信息。
- 可操作性：每一条都是具体、明确且可执行的指令。



- 脆弱性测试：每一条若被删去，都会直接导致Claude犯错或偏离预期。

在第3章中，我们将正式踏入Skills的广阔世界。

如果说CLAUDE.md是赋予Claude基础认知的“入职手册”，那么Skills则是为其量身定制的“专业培训教材”。

当你的“入职手册”变得日益臃肿，试图将庞杂的领域知识全都塞入其中时，不仅会导致上下文资源的浪费，更会降低核心规则响应效率。此时，你就需要一种更优雅、更智能的按需加载机制，这正是 Skills系统的核心价值：让专业知识在真正需要的那一刻，精准地注入对话上下文中。

## 思考题

1. 请梳理当前项目中Claude必须知晓的核心信息。试着运用“三问框架”(WHY、WHAT、HOW)进行拆解，并据此撰写一份不超过30行的CLAUDE.md。成稿后，请逐行审视：如果删去此行，Claude会犯何种错误？若你无法回答这个问题，则说明该行内容或许是多余的。
2. 在团队协作中，若开发者针对CLAUDE.md的规则产生分歧（如缩进采用2空格还是4空格），应如何利用“五层记忆体系”来化解？请具体阐述：哪些规则必须置于项目层以强制统一标准，而哪些偏好又适合下放至用户层或本地层，从而保留个人的自由选择空间？
3. 请在你的项目中执行 `/init` 命令以自动生成CLAUDE.md，随后将其与手动编写的版本进行深入对比：机器能够准确推断出哪些显性信息？而又有哪类“隐性知识”必须依赖人工补充？